How-To Library

Informatica

# Replacing Complex Patterns in Source Code for MetaDex Scanners

© Copyright Informatica LLC 2022, 2023. Informatica and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html.

# Abstract

You can search and replace patterns of text in scanner input code. Use a JSON configuration file to specify the patterns that you want to replace and define the code that you want to replace them with. Configure the path to the replacement file in the MetaDex tool.

Read this article to learn how to configure a JSON file with replacement patterns.

# Supported Versions

- Enterprise Data Catalog 10.5

# Table of Contents

Overview. . . . . 2
Configuring the pattern replacer utility. . . . . 3
Configuration parameters. . . . . 3
Example configuration. . . . . 5

# Overview

The pattern replacer utility searches for patterns that you specify in a JSON file and replaces them with code that you define in the file. Configure the path to the JSON file in the MetaDex tool.

With the utility, you can modify input code when it contains unsupported syntax or complex structures that a scanner can't parse. You can also use the pattern replacer when the source code contains highly dynamic code and code written with non-standard tools.

**Important:** Use caution when modifying the code with the pattern replacer. Modifications that distort the code or change code logic can lead to unexpected results.

You can process input for the following scanners and functionalities:

- Azure Data Factory scanner
- BTEQ scanner
- COBOL scanner
- Databricks evaluator
- Databricks Notebooks scanner
- Database Objects scanner
- Oracle SQL Loader
- Oracle SQL Plus scanner
- PL SQL script
- SAS Enterprise Guide projects scanner
- SAS logs extractors
- SAS scanner
- SQL Evaluator module

  **Note:** If the MetaDex configuration includes the SQL Evaluator module, you can use the pattern replacer from the scanner. For example, to use the pattern replacer for SQLs from Microsoft SQL Server Integration Services,

use the **PatternReplacer configuration path** parameter. To access the parameter, select the **Expert** mode in the MetaDex tool.

- Teradata MultiLoad scanner

# Configuring the pattern replacer utility

To configure the pattern replacer, you first create a JSON file with the replacement values. Then, configure the path to the replacement file in the **Pattern Replacer configuration path** field, available in the MetaDex tool.

1. Create the JSON file.
   a. Create a JSON file with "replacementRulesList" as the top-level element.
   b. Make sure the path to the file follows the URI convention.
   c. Configure the file.
2. Link the JSON file with your configuration in the MetaDex tool.
   a. Open your project.

      The project page appears.
   b. From the mode list, select the **Expert** mode.
   c. In the **Reader** mode, go to the **PatternReplacer configuration path** field.
   d. Enter the URI to the JSON file.

## *Configuration parameters*

When you configure the JSON file, use elements and variables that determine how the utility executes the search and replace.

### JSON file elements

The following table describes the elements that you configure in the JSON file to replace patterns within the source code:

| Element name | Description |
|---|---|
| replacementRulesList | The main JSON object that contains a replacementRulesList field of the list type. |
| condition | The condition object governs the application of the rules. The condition object is of the text type. You can use variables in a condition. The variables vary, depending on the scanner. The utility evaluates conditions for each input. If the evaluation returns TRUE, the pattern replacer applies the changes to the input text. |
| instructionList | The instruction List contains a list of patterns fo find and replace. |
| pattern | A text or object type field. It represents a Java regular expression. If there are backslashes in the pattern, ensure that you replace \ with \\ . The pattern field can be a single string. MetaDex replaces the input text with the CASE_INSENSITIVE Java Pattern. |

| Element name | Description |
|---|---|
| replaceWith | A text type regex string that replaces the pattern. |
| regex | Contains a Java regular expression string. |
| mode | Optional. Represents the mode of regex matching. The mode can have one of the following values:<br>- SINGLE LINE<br>- MULTI LINE |

## Condition variables

The following table lists the scanners and the available variables:

| Scanner or functionality | Available variables |
|---|---|
| BTEQ scanner | - SourceLocationFilename |
| COBOL scanner | - SourceLocationFilename |
| Databricks evaluator | - SourceLocationFilename |
| Database Objects reader | - SqlProcedure<br>- SqlPackage<br>- SqlView<br>- SqlExternalTable<br>- SqlHost<br>- SqlDatabase<br>- SqlSchema<br>- SourceLocationFilename |
| Microsoft Azure Data Factory scanner | - ActivityDataFlowLocationName |
| MultiLoad scanner | - SourceLocationFilename<br>- SourceLocationFilePath |
| SAS | - SourceLocationFilename |
| SAS EGP scanner | - SourceLocationFilename |
| SAS logs extractors | - SourceLocationFilename |
| SQL Evaluator module | - SqlHost<br>- SqlDatabase<br>- SqlSchema |
| SQL Loader scanner | - SourceLocationFilename<br>- SourceLocationFilePath |
| SQL Plus scanner | - SourceLocationFilename |
| PL/SQL script | - SourceLocationFilename |

## Example configuration

Configure the utility to replace the source table T with the source table R in a single line query.

Use this example if you want to replace the source table T with the source table R in a single line query. The query starts with "SELECT" and ends with "FROM T". This example shows how to replace the source table for the "SQLSERVER" host, the "tempdb" database , and the "dbo" schema. Enter the following sample configuration in the Pattern Replacer JSON file:

```
{
  "replacementRulesList": [
    {
      "condition": "${SqlHost} == 'SQLSERVER' and ${SqlDatabase} == 'tempdb' and ${SqlSchema}
== 'dbo' and ${SqlProcedure} == 'myProcedure'",
      "instructionList": [
        {
          "pattern": "(SELECT .* FROM) T",
          "replaceWith": "$1 R"
        }
      ]
    }
  ]
}
```

## Authors

**Ewa Pydych**

**The author would like to acknowledge Tomasz Bińczycki and Tomasz Czeleń for help with this article.**